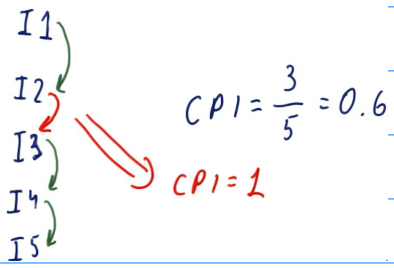
ILP

Branch prediction and IF conversion, help us eliminate most of our pipeline problems caused by control hazards. But data dependencies also can prevent us from finishing one instruction per cycle. What can we do about data dependencies, why only limit at 1 instn. per cycle. In this chapter we will see about Instruction level parallelism, which tells us how many instns could possibly be executed in any given cycle.

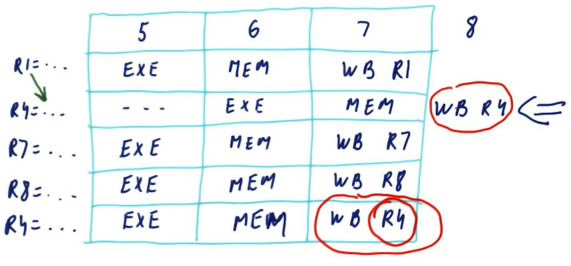
Even with very large no. of execution units , we can't achieve 1/#exe_unit CPI because of data dependencies(RAW dependencies) . Say we have an ideal processor with infinite # of execution units, we still not gonna get CPI of zero because, of Dependencies.

RAW DEPENDENCIES



Assume if all instns are independent, then our CPI would be 0.2, neglecting the time to fill the pipeline.

WAW DEPENDENCIES



In order to write to R4, in the same seq. as programmer intended, the processor needs to figure out a way to write to R4 after cycle 8 i.e is cycle 9 (for performance reasons, write asap).

Another thing is we are able execute I2 in 6th cycle only before result of I1 been written in R1, because forwarding is implemented. So if prev. instn is alu, we can stall just one cycle, and for LD, 2 stalls.

DEPENDENCY QUIZ

- 5-STAGE (FETCH, RR, EXE, MEM, WREG)
- FORWARDING!
- 10 INSTS IN EACH STAGE

WHEN DO WE		EXE	WB:
MUL	R2, R2, R2	2	4
ADD	R1, R1, R2	3	—
MUL	R3, R3, R3	—	—
ADD	R1, R1, R3	—	—
MUL	R4, R4, R4	—	—
ADD	R1, R1, R4	—	—

	5
2	4
4	6
2	4
5	7

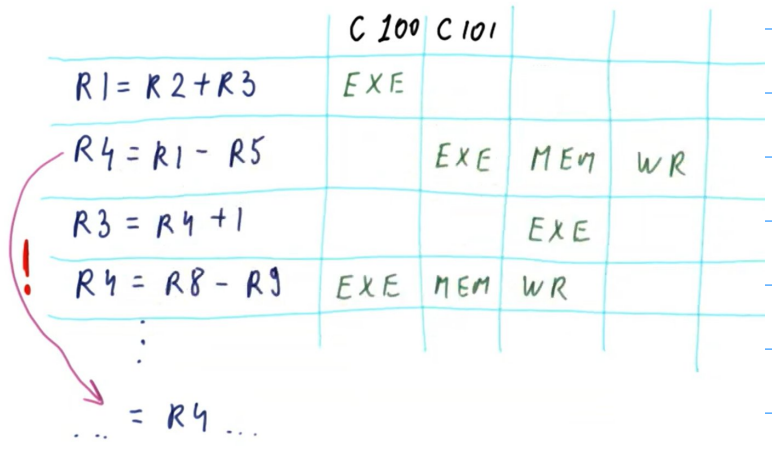
REMOVING FALSE DEPENDENCIES

RAW — TRUE DEPENDENCIES

WAR, WAW } FALSE (NAME) DEPENDENCIES

False dependencies exist just because, we use same registers for two diff results. For RAW dependencies, we must delay executn, where as for false dependencies, we can execute , just that we need to take care of multiple values for that register.

DUPLICATING REGISTER VALUES



If we were to duplicate the regs we need to maintain multiple versions of REG array. And the instn need to see the register, if it exists (valid) in multiple versions, the instn needs to select lastest version. Which is complicated..

Register renaming is another scheme, relief from false dependencies, but in a way that is more managable.

REGISTER RENAMING

ARCHITECTURAL REGS = REGS PROGRAMER/COMPILER USE

PHYSICAL REGISTERS = ALL PLACES VALUE CAN GO

REWRITE PROG TO USE PHYSICAL REGS

REGISTER ALLOCATION TABLE (RAT) = TABLE THAT SAYS WHICH PHYSICAL REG HAS VALUE FOR WHICH ARCH REG

As the processor fetches the instn. and decodes them it also does reg renaming, which means it rewrites the program to use phy regs.

The processor knows which arch. reg is placed in which physical locations through a table called RAT. So for each of the arch regs, there will an entry in the table which says what is the current physical locations from where we can find that arch reg's values.

RAT EXAMPLE

ADD	R1, R2, R3	ADD	P17, P2, P3
SUB	R4, R1, R5	SUB	P18, P17, P5
XOR	R6, R7, R8	XOR	P19, P7, P8
MUL	R5, R8, R9	MUL	P20, P8, P9
ADD	R4, R8, R9	ADD	P21, P8, P9

... R4 ... =) ... P21 ...

RAT

0	P0
1	P17
2	P2
3	P3
4	P21
5	P20
6	P19
	...

Quiz Sol:
Mul P7, P2, P2 & RAT R2=P2→P7
Add P8, P1, P7 & RAT R1=P1→P8
MUL P9, P4, P4 & RAT R2=P7→P9
ADD P10,P3, P9 & RAT R3=P3→P10
MUL P11,P6, P6 & RAT R2=P9→P11
ADD P12,P5, P11 & RAT R5=P5→P1

REGISTER RENAMING QUIZ

FETCHED	RENAMED
MUL R2, R2, R2	
ADD R1, R1, R2	
MUL R2, R4, R4	
ADD R3, R3, R2	
MUL R2, R6, R6	
ADD R5, R5, R2	

RAT

R1	P1
R2	P2
R3	P3
R4	P4
R5	P5
R6	P6

Quiz Sol:
Mul P7, P2, P2 & RAT R2=P2→P7
Add P8, P1, P7 & RAT R1=P1→P8
MUL P9, P4, P4 & RAT R2=P7→P9
ADD P10,P3, P9 & RAT R3=P3→P10
MUL P11,P6, P6 & RAT R2=P9→P11
ADD P12,P5, P11 & RAT R5=P5→P12

RAT
R1 P8
R2 P11
R3 P10
R4 P4.
R5 P12
R6 P6.

We have output dependence between all the MULs (#2) and anti-dependence between add-mul pairs(#2). They are absent in the renamed code, while true dependencies exist.

For our renamed program, we can execute all the MULs in 1st cycle and rest in second cycle. Because anti dependencies (War) and waw dependencies no longer there.

In the initial program, if we look at all the dependencies (raw & anti), basically its a sequential program.

FALSE DEPENDENCIES AFTER RENAMING?

FETCHED	RENAMED
MUL R2,R2,R2	MUL P7,P2,P2
ADD R1,R1,R2	ADD P8,P1,P7
MUL R2,R4,R4	MUL P9,P4,P4
ADD R3,R3,R2	ADD P10,P3,P9
MUL R2,R6,R6	MUL P11,P6,P6
ADD R5,R5,R2	ADD P12,P5,P11

FETCHED:
CPI = 1 (IPC = 1)

RENAMED:
CPI = $\frac{2}{6}$ = 0.33 (IPC = 3)

INSTRUCTION LEVEL PARALLELISM (ILP)

ILP = IPC WHEN:
- PROCESSOR DOES ENTIRE INST IN 1 CYCLE
- PROCESSOR CAN DO ANY NUMBER OF INSTS IN SAME CYCLE
  - HAS TO OBEY TRUE DEPENDENCIES

STEPS TO GET ILP:
1. RENAME REGISTERS
2. "EXECUTE"

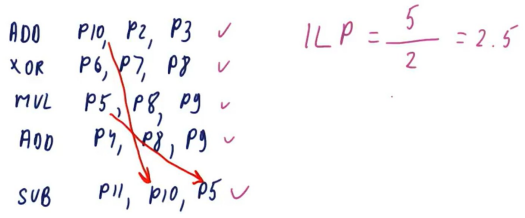
ILP IS A PROPERTY OF A PROGRAM
NOT OF A PROCESSOR

We define ILP as what would this program do as far as the ILP is concerned on an ideal processor(single cycle, non pipelined , large # of cores). So it doesn't depend on what processor we actually run on. It has everything to do with just the dependencies of the program.

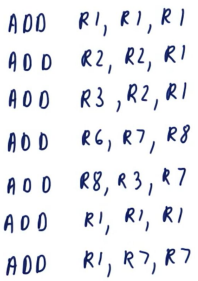
We can't say ILP on a processor, but we can say ipc of a program on a processor.

While computing the ILP of a program, we need not actually rename, we can just look at the raw dependencies only, because other two types of dependencies are resolved by renaming.

ILP EXAMPLE



ILP QUIZ



ILP=?

7/4



ILP only considers true/flow/RAW dependencies

ILP WITH STRUCTURAL & CONTROL DEPENDENCIES

- NO STRUCTURAL DEPENDENCIES
- PERFECT SAME-CYCLE BRANCH PREDICTION

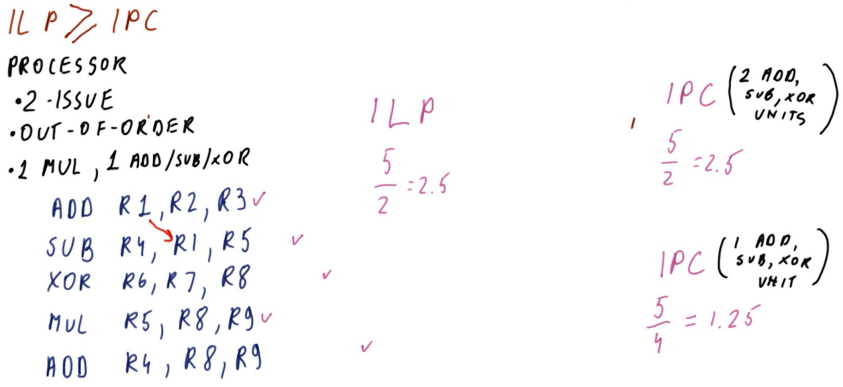


There are no structural dependencies. SD occur when we don't have enough hardware. ILP is when program is on ideal hardware, so no SD.

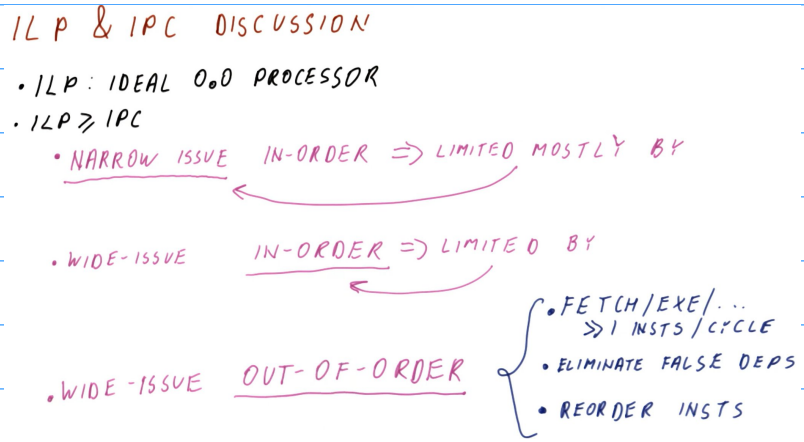
Regarding control dependencies, we assume perfect prediction, meaning, we predict it and execute even before branch is executed/ resolved.

So pretty much we can ignore ctrl dependencies. Even if the branch might get delayed due to raw dep. , but instns. that have ctrl dependency on it will continue. As soon as we begin fetching, we knew exactly which instns will be executed.

ILP is IPC on a perfect processor that does perfect branch prediction and only has to obey data dependencies



On ideal proc : second instr. is on 2nd cycle due to raw, so ipc is 2.5
On proc config1: second and last instr. is on 2nd cycle due to raw & structural dependencies respectively, so ipc is 2.5 again
On proc config2: 2nd I in 2nd Cycle, 3rd I in 3rd C, last I in 4th Cycle

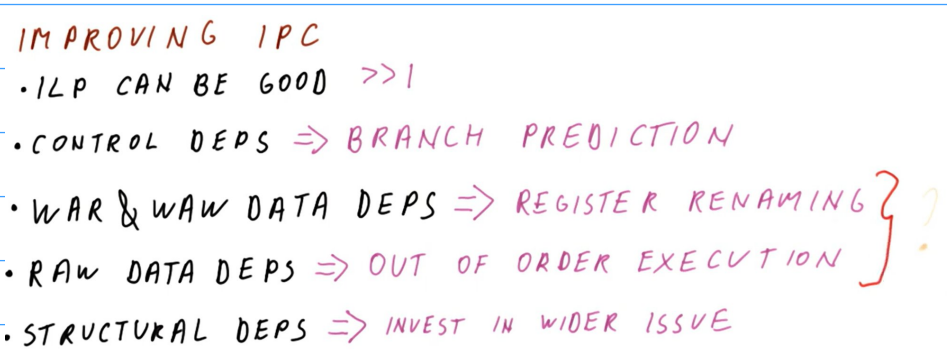


An out of order processor, can issue something like 4 or greater instns per cycle.

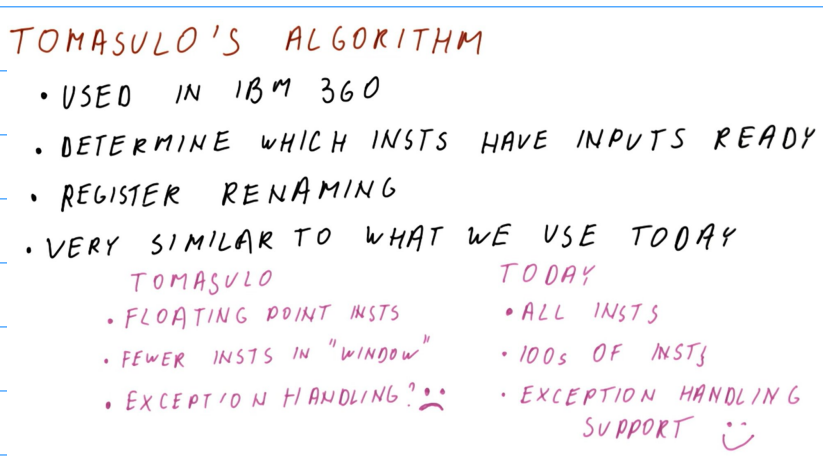
In this (ILP) lesson we have seen we can get excellent performance even if we have lot of data dependencies in the program. In the nxt lessons, we will learn how a actual processor do this only with hardware structures that have limited size and complexity.

Tomasulo Algorithm

In this lesson we will see how instn. scheduling can let us execute instns faster even when programs have data dependencies and also how to exe more than one instn every cycle.



ILP typically is way above 4. Among the listed, we need to figure out how to do last 3. But we will focus on 3rd and 4th. How do we do Reg renaming and OoO execution in a way ameanable to hardware implementation.



It is one of the first techniques in OoO execution its more than 50 yrs old. It also includes a form of register renaming. Its very similar to today's OoO execution algo.

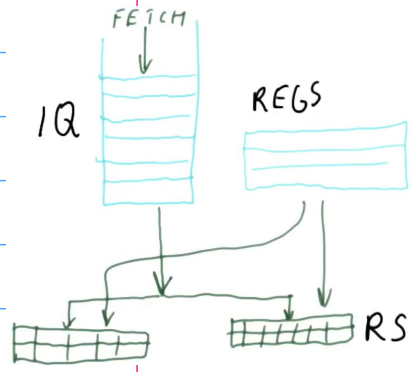
In olden days exception handling was not supported, but that was ok, in case of exceptions they would do some drastic measures. But today its explicitly supported.

First we will see how tomasulo's algo did it, later we can see what extra things are done in modern processor.

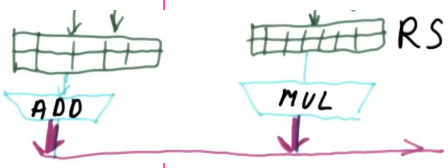
Before we dive deep into inner workings of tomasulo's , an overview of whats happening there.



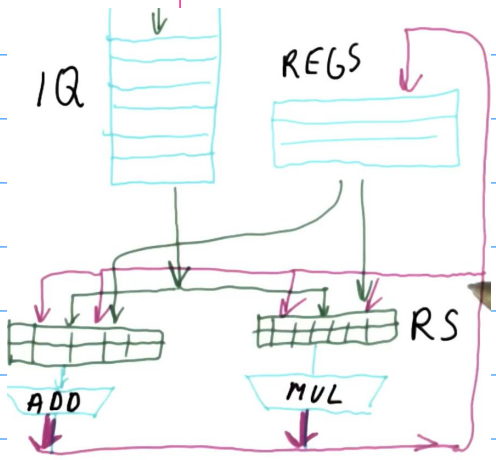
We have the Instn. Queue which is a fifo that holds FP instructions as they are fetched. We grab the next available instruction from the IQ and then put into one of reservation station.RS is the place where instns wait for their parameters to become ready.



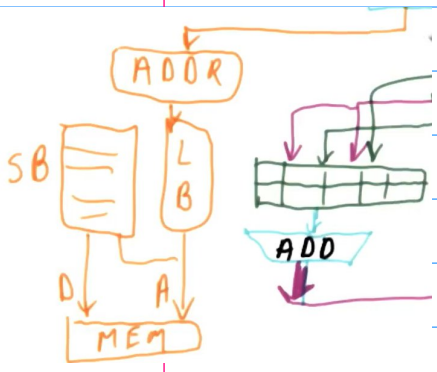
There is a FP REG file. When an instn inserted into RS, the values in Reg file that are ready for execution, are gonna enter the RS.



Once an instn. is ready to execute, it goes to execution unit. Each execution unit has separate RS. Once executn unit produces a result, the result will be broadcast, on a bus. So ofcourse the result would go to reg file, from where instns in RS are gonna grab values .But this results are also broadcasted to the RS, for implting data forwarding.

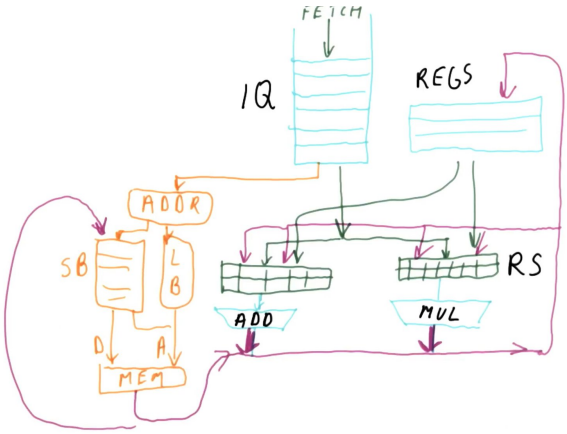


When waiting, some values may be already available from reg file, and instns might be waiting for some values that are yet to be executed. Once results are available, they are broadcasted, we check if this is the one we are waiting for, by comparing tag, and sample it .

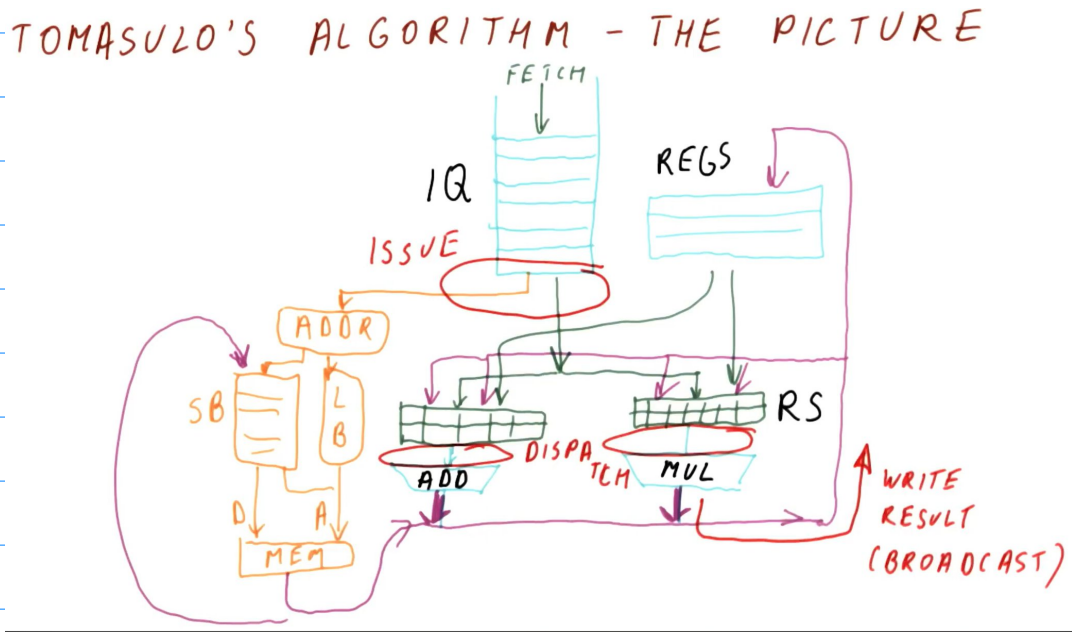


And finally, if the instr. is not arithmetic instn, if its a load/store to/from the FP reg file, then its gonna go to addr. generation unit which is integer operation so its not gonna go into FP exe units. So basically there we compute the addr. and push the instn into the load buffer or a store buffer where the instns are gonna queued up for going to mem.

LB gives only A, whereas SB gives both A and D, when a load comes back from mem, its value is also broadcasted on the CDB . CBD also goes to SB, so that stores can get their values when they become available. Until it becomes available, its gonna wait in SB. But keep in mind ld/st instns can't execute in OoO in this machine unlike Arithmetic instns. So basically we don't reorder loads and stores. Modern processor does that reordering of lw/sw but not tomasulo's.



Some terminology: The process of sending instn. from IQ to RS is called issue. And from RS to EXE_UNIT is called dispatch. And EXE done and result available is called broadcast.



Tomasulo's ISSUE

*Issue needs to done in program order for reg renaming to work correctly.

*Determine where the inputs come from , using some sort of RAT.

*Get free reservation station (of correct kind)

*Put instn. in RS

*Tag the destn. reg of instns , so that results when they are produced, goes to required instructions in RS.

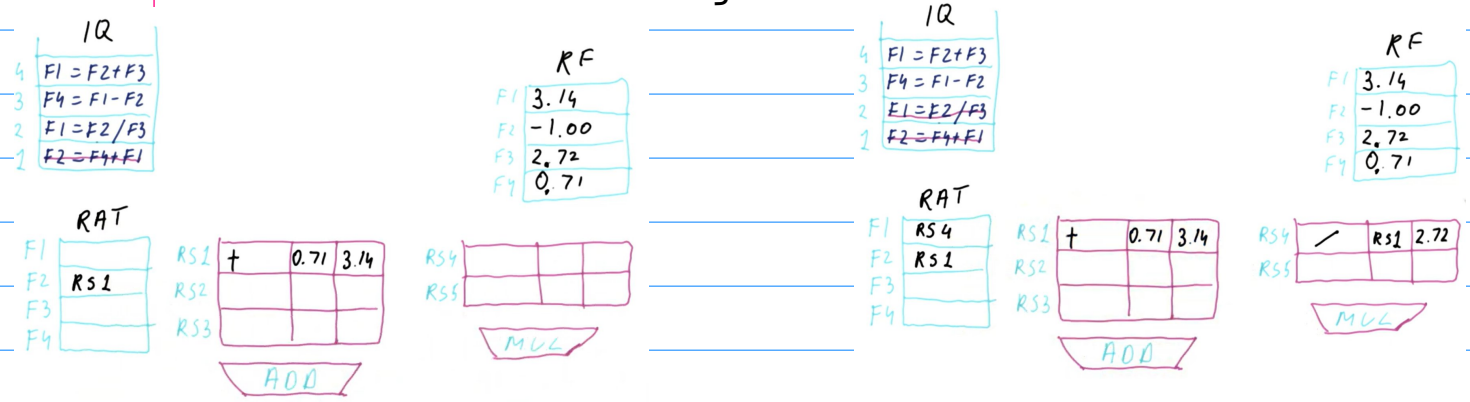
ISSUE EXAMPLE



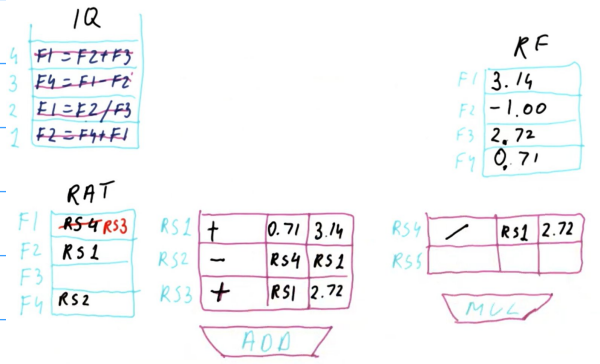
When the Reg Alias Table is empty, we can read directly from RF, we need not wait for any results to be produced.

The steps are:

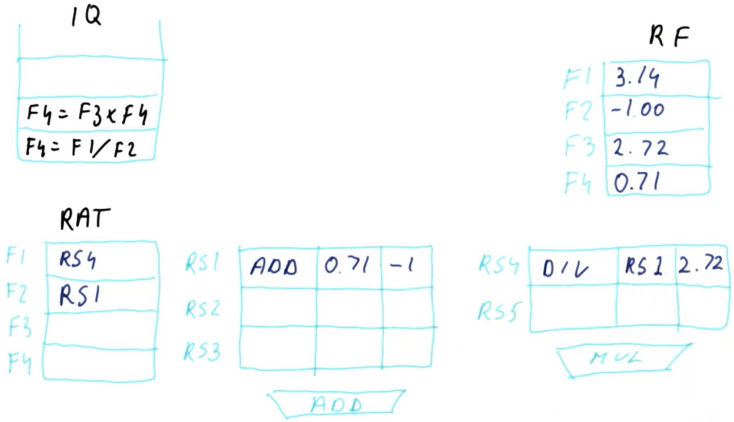
* Take the instruction from the instn queue, and look in the RAT to see if our inputs are ready or not.
* If regs available, read values from RF and get into RS(if available , if not available , issue is stalled).
* And the final step for Reg renaming is we gonna say that the result of instn is gonna found in RSx , by mentioning that in the RAT, So future instns are gonna wait for result of RS1 after its exe



In a real proc. , we would issue one instn. each cycle, and by that time, the instructions in RS would have executed, but here we are seeing , they are not executed for some reason , just to illustrate in example.



ISSUE QUIZ



Issue whatever instns that can be issued, assuming the instns not leaving RS , for whatever reason.

Steps:

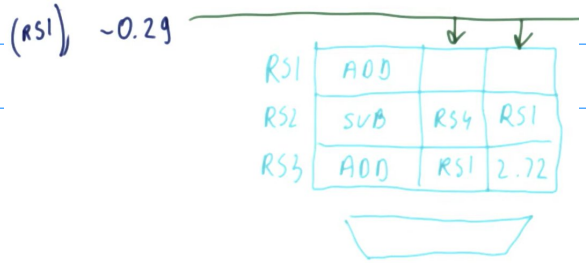
* Issue DIV, update RS5 as "DIV RS4 RS1", as there is RS available
* Update RAT's F4 entry as RS5

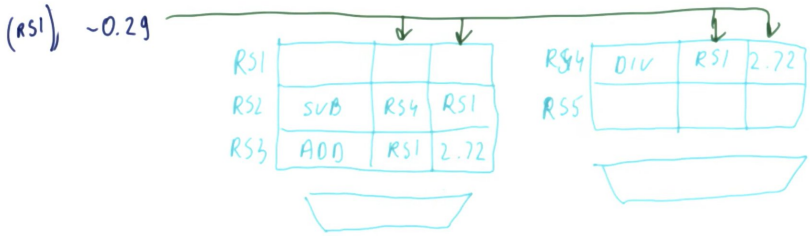
We cant issue MUL instn, because reservation stations are filled.

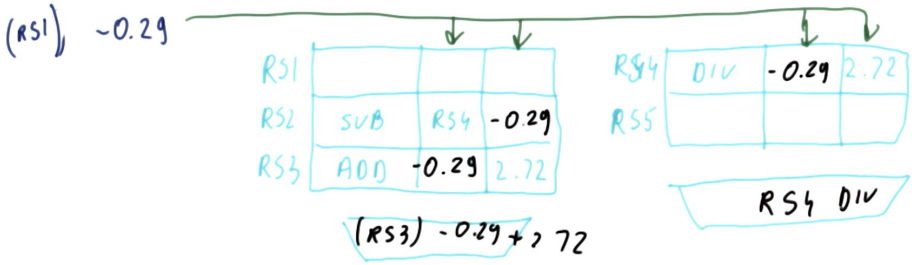
Dispatch

Dispatch basically needs to consider latching operation's results that are produced, and also deciding which instns to execute. In the same cycle we will have both of them.

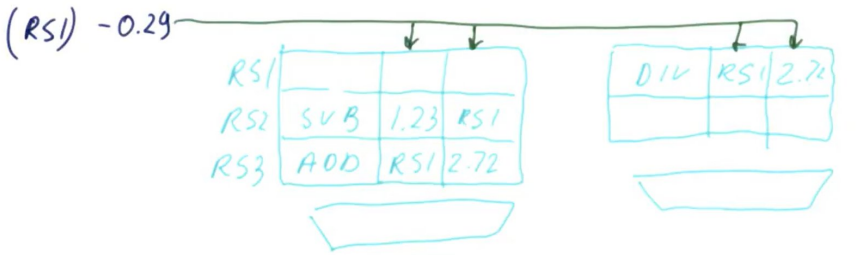
When we broadcast the result we say the value , as well as which RS is producing it. Once broadcasted, we are gonna free the RS who is producing that result, because it is done executing. The nxt thing we do is match result's RS tag with all the intn's 2 operands in all the RS







During dispatch, when more than one instn is ready..



We would want to execute that instns which will enable us to execute more future instns. so as to maximize performance . But we can't see into future, which is not possible in hardware as we can see in RS and possibly in the IQ. So basically we have few heuristic options.

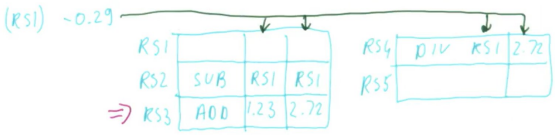
* Oldest first, because there can be chance that for older instns there are more instns waiting for it compared to newer.
* Most dependent ones first. We are gonna search RS's operands , but this requires lot of searching, so power hungry.
*Random would also work because if we select randomly, then at some point if the most dependent is not getting executed, all other instns will be dependent on it, so eventually it will run. But worst perf of all these three.

So its not which is more correct, all will work, but which will work reasonably well . SO oldest first typically helps somewhat.

DISPATCH QUIZ

WHY HASN'T RS3 DISPATCHED BEFORE THIS?

- IT WAS ISSUED IN THE PREVIOUS CYCLE
- ANOTHER INST WAS DISPATCHED TO ADD UNIT
- RS2 IS OLDER THAN RS3, SO RS3 CAN'T DISPATCH UNTIL RS2 DOES



Say the state at beginning of the cycle is like this. The instn is not waiting for anything then why has it not yet dispatched. Select all that can be possible.

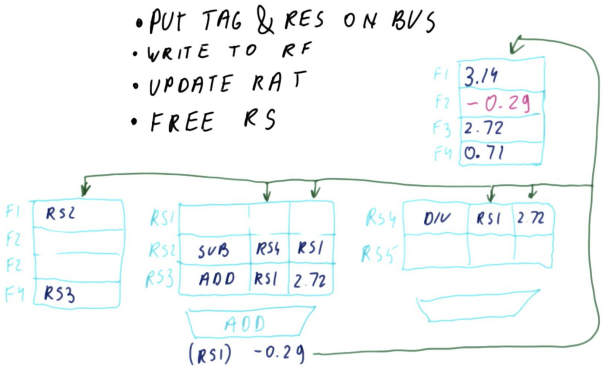
We are broadcasting RS1 result, and state of RS is as shown. RS3 is not waiting for anything at this point, then why has this not dispatched in the previous cycle before we got into this cycle.

1st option says rs3 got its entry inserted in the previous cycle.
2nd option says another instn got dispatched, so this one couldn't

Solution is 1 and 2, not option 3 because execution in OoO is the very nature of OoO processors, and as tomasulo's is an OoO algorithm, 3rd option can't be true.

WRITE RESULT (BROADCAST)

- PUT TAG & RES ON BUS
- WRITE TO RF
- UPDATE RAT
- FREE RS



The final step in tomasulo's is wr result/ broadcast step. The steps here:

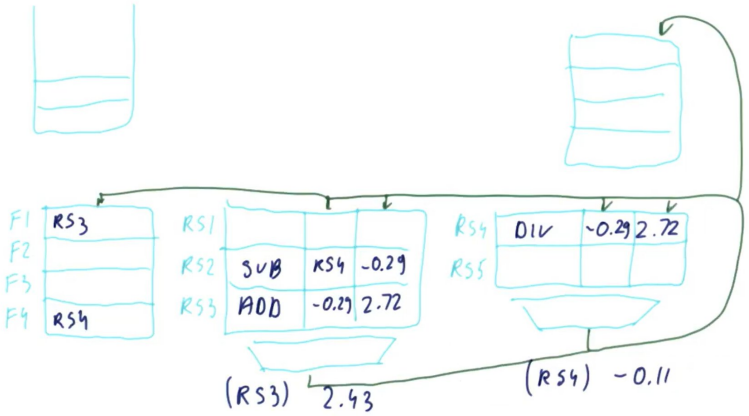
Put the tag(rs-tag) and result on broadcast bus.

write the result on to the RF by seeing the rat, which register corresponds to that RS tag.

update the rat, to point to rf, instead of rs-tag, basically we invalidate

invalidate the corresponding RS

BROADCAST >1 ?

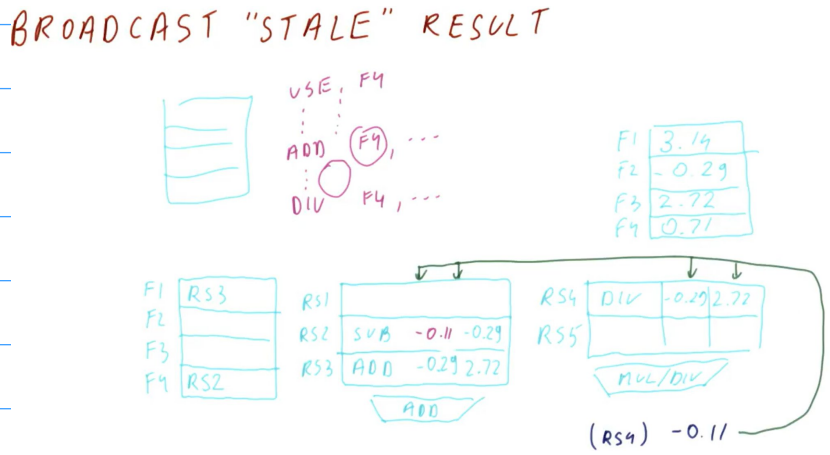


What if we have several instn results to broadcast. There are many ways to handle this in hardware, have separate bus for each exe unit, Then we would have twice as many comparators for each operands in each RS for tags similarly we would have to have two results at RAT and in RF.

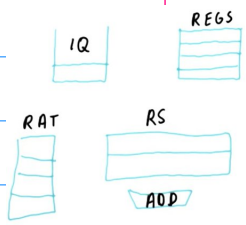
If we have only one bus, which one do we broadcast? one approach is to have priority on one unit than other, usually slower unit(mul/div) is given priority over faster one, that is because, as it is slower, we may have more instns piled up waiting for this result in the RS. Its a kind of age based thing we had in RS multiple dispatch problem, but here instead of having some metadata to tell the age, we simply prioritize slower exe unit, this is a very common heuristic. IF we want to do better than that, we can carry age of instn in RS, which amount to almost the same thing, because slower execution unit's instns will be older. Or we can do something more adv. but typically we choose slower exe unit to have priority and this is common heuristic.

Now the question is what if we broadcast a stale result. Stale result is the result we are broadcasting , we dont have any entry in RAT having this RS. In that case we dont touch RAT and RF.

If the tag that is being broadcasted, doesn't match any entry in RAT, we dont update RAT and RF because all of the instns that possibly need this value are already in the RSs (see example in pink, top instns are newer, as its a queue).

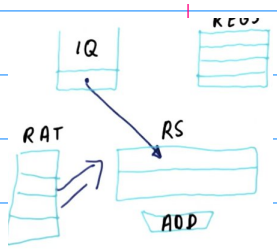


Now that we have seen all the steps in tomasulo's, lets review it..



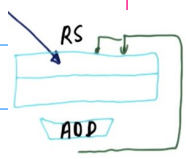
If you look at steps for one instn:

* We take the instn in the IQ and issue it to a reservation station and we look-up the rat to tell us where the operands gonna come from.



* Now that instn is issued, it sits in the RS and waits for operands to become ready, during that time instn is trying to capture the results broadcasted by other instns

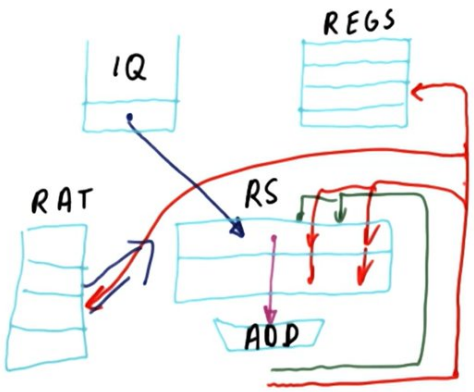
* Once we capture the last operand that we are missing, we dispatch



* And when its done exe, it broadcasts results, which is used by other instns in RS to capture their missing operands, it also writes to register that it is supposed to update, and it updates the rat for that register making that entry as invalid, so that future instns look in RF, and not be waiting for this instn to complete anymore.

* All of the above activity is happening for every cycle, just that it is happening for diff instns. Because each of this steps can happen everycycle, there are some intresting things that we need to consider.

TOMASULO'S ALGORITHM - REVIEW



FOR ONE INSTRUCTION:
- ISSUE
- CAPTURE
- DISPATCH
- WRITE RESULT

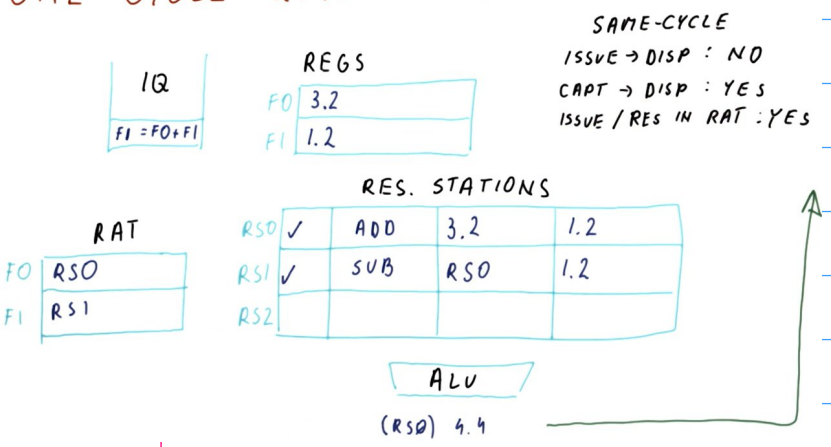
EVERY CYCLE:
- ISSUE
- CAPTURE
- DISPATCH
- WRITE RESULT

* same cycle issue dispatch, if it doesn't need to capture anything ? NO During issue, its considered being written in RS, so not possible to dispatch. But it is possible to design proc. in such a way that allows same cycle issue and dispatch
* same cycle capture-dispatch, typically the answer to this is also NO, because the RS updates its status to become valid for dispatch (operands missing to operands available) during the cycle it captures. so in nxt cycle only we can dispatch. But it is possible to do in same cycle, it just needs more hardware for that.
* Can we update RAT for ISSUE and WR_REGS in same cycle. (So if the dest reg of instn being issued and the dest reg of instn thats broadcasting is same, then we need to update the RAT twice.) the answer is YES. Because we can give priority to ISSUE and write its RS tag into RAT and the instn thats broadcasting will be captured by instns already waiting in the RS and all future instns in IQ will get it from RS. Writing instn is tring to point RAT to register that its going to write, but its not correct because newer instns need the latest result So we prioritize issuing instn's write request when both need to be written.

NEED TO KNOW IF:
- SAME-CYCLE ISSUE → DISPATCH NO
- SAME-CYCLE CAPTURE → DISPATCH NO
- UPDATE RAT ENTRY FOR ISSUE & WR.RES ? YES

- DISPH
- WRITE

ONE CYCLE QUIZ PART 1

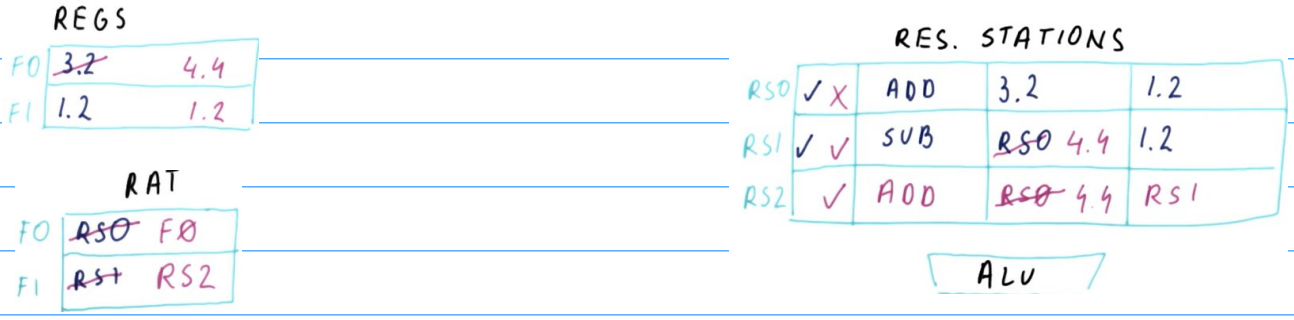


Shown is the state at the beginning of the cycle, after the end, what is the contents of RAT and RF.

Sol: F0 will be invalidated in RAT and in RF, F0 stores 4.4

Part2: what is in RS after the end of this cycle.

Sol: RS2 gets updated, with ADD 4.4 RS1(we see in RAT and fill values)
RS0 gets invalidated
RS1 becomes SUB 4.4 1.2 (& will be dispatched)



part3: What all will dispatch if any in this cycle.

Only RS1 will dispatch after it captures the RS0 result

Note that even if RS2 captures results it wont dispatch because ISSUE-DISPATCH can't happen in same cycle.

Which all is not true about tomasulo's??

1. It issues instns in program order

2. It dispatches in program order

3. It writes results in program order

Option two is incorrect.

Option three is also incorrect as it is not necessarily writes in program order, it writes(broadcasted) in the order as the results are produced

Now lets briefly consider what happens to load and store instn.s

Just like we have data dependencies that go through registers, there can be dependencies through memory. Lw/Sw are the only instns that can have dependencies through memory.

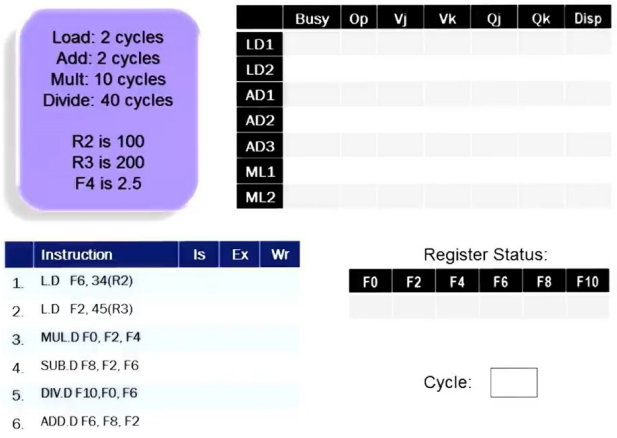
* RAW: SW to A then LW from A
* WAR: LW then SW
* WAW: Two stores to same location

So what do we do

We can do in order. In tomasulo's we do IN-ORDER so no problem

Another option is , identify dependencies and reorder them, like we do for normal instns., but it turns out to be more complicated than reg based dependencies. Modern processors do this. We will consider later how that is done.

TOMASULO - LONG EXAMPLE



The characteristics of the processor are:

the load unit takes 2 cycles, add unit takes 2 and so on..

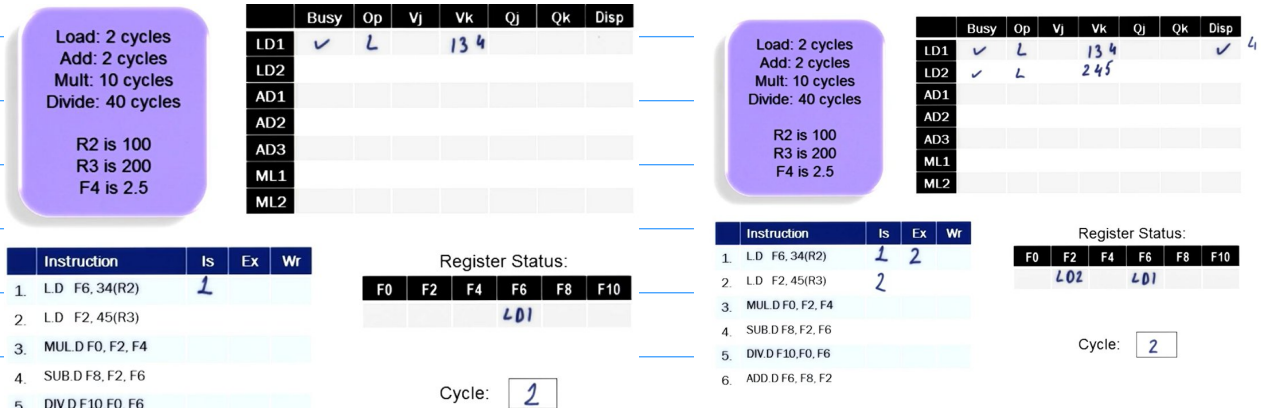
Instr. queue is given. , in that table we will see in which cycle the instr. is issued, begin exe and write results.

RAT is given. Cycle number can be written.

RS is also given, it tells if RS is busy , what operation it does, values of operands if available, else Q columns hold tag values that we are waiting for.And last column tells if the instn is dispatched.

The load reservation stations are basically load queues, so they are gonna be slightly diff from the other RSs.

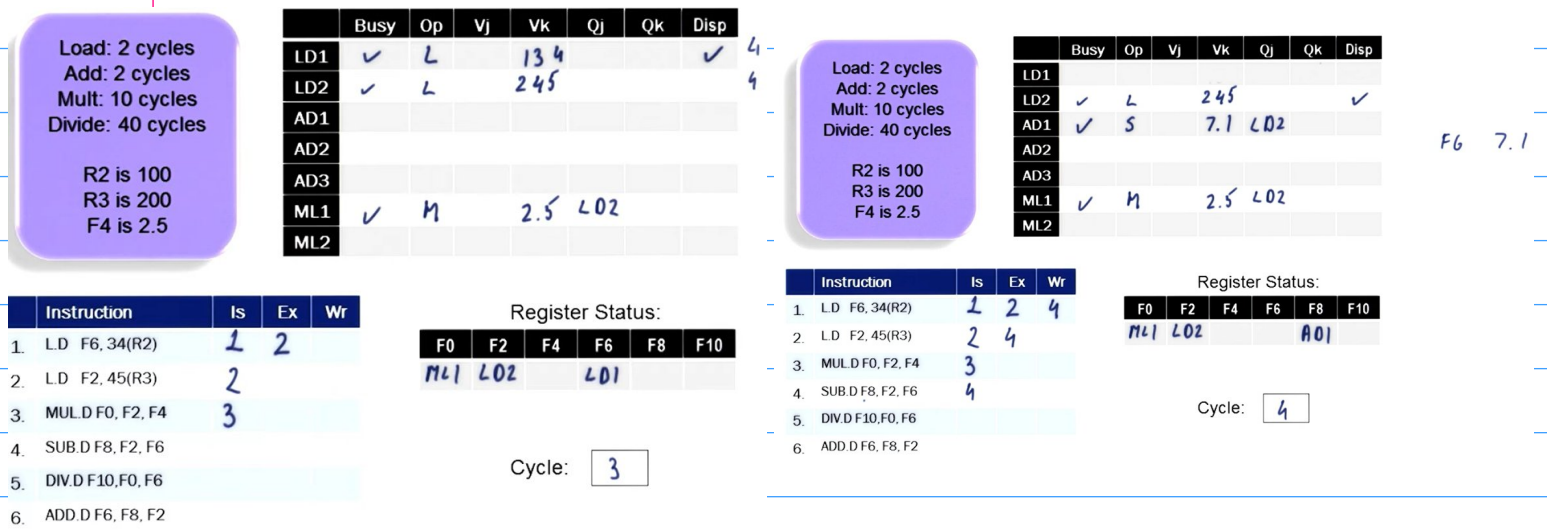
Lets begin with cycle 1, there is nothing to dispatch and nothing to write So the only thing that we need to be concerned is if we are issuing any instr. The LD sees the LD ST queue, and we have space there, so we issue. We have imm addressing, and it will be computed before we put into RS. This load will produce a FP value, so put that RS station in the RAT. We cant dispatch issued instn. because we cant do issue-dispatch in same cycle.



In the second cycle, we dispatch the next load following the same procedure as above. Now we can also dispatch the load in LD1. So there is an assumption to make here, load's execution takes two cycles, now wr_result happens in the same cycle or does it happen in the nxt cycle?, here we assume wr_result happens in cycle after execution has done. So if execution happens in cycle 2, and wr result happens in cycle 4, we mark it aside the first table entry just not to forget in cycle 4.

So now let's look at cycle 3, here we try to issue the mul instn, we have RS, so we issue. Second operand is read from RF and first operand we mark as we are waiting from LD2. Then we are gonna rename RAT's F0 to point to ML1 RS. In this cycle the first load is still executing, the q. is can 2nd load begin execution. If we have a pipelined Ld/st unit, we can execute next load, but say its not pipelined, and so we can only start exe of 2nd load after 1st has finished.(say only our add unit is pipelined). We can't dispatch ld2 until cycle 4. We will mark that we can dispatch in cycle 4.

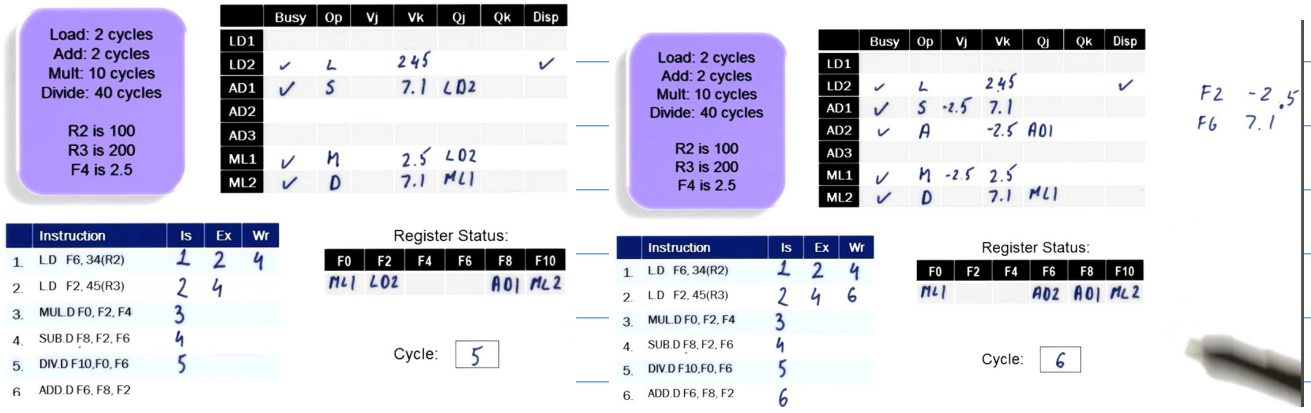
We can't exe ML1 as its waiting for its operands. Nothing is broadcasting its results. LD1 is still executing.



In cycle 4, We will try to issue the nxt instn. which is subtract, and we have free RS, so we put in AD1. We will be waiting for LD2 and LD1. And we will mark F8 as renamed to AD1 in RAT. This completes the ISSUE in cycle 4 Now lets see if we can dispatch something. In cycles 2 and 3, we were exe first load, and now its ready to write. So we can issue 2nd load. Are any other ready to execute? well not in this cycle. So now we are gonna write the result in LD1. We are gonna capture that value in all the places that are waiting for LD1 in RS. The sub thats issued in this cycle captures it. Say the value is 7.1 . Then we put that 7.1 in the RF and invalidate the RAT. And also invalidate the LD queue that holding the first load. In cycle 4 we have done issue, dispatch (found only LD can be dispatched) , write back (only one result can be written anyway). So now we look at 5.`

In cycle 5, we are gonna issue div, because ML2 RS is available. We are gonna look at RAT to see where our operands come from. We can read F6 from RF, and F0 will wait for ML1, then we are gonna rename F10 in rat to point to ML2's result. That completes the issue for this cycle.

Now as far as the dispatch is concerned, we have already dispatched this load(LD2), and nothing else have their all operands ready. And there is no result that is gonna broadcasted in this cycle.



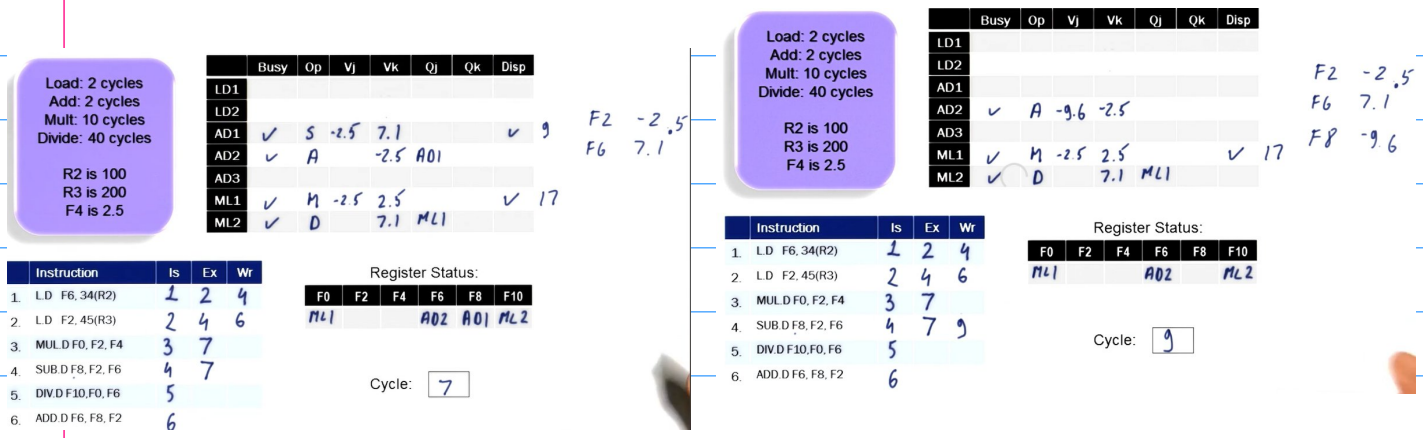
In cycle 6, we will try to dispatch ADD, and because there is unit available, we will suceed in dispatching it. F8 and F2 operands will be waiting for AD1 and LD2 RSs. Rename F6 to point to AD2. That concludes issue. Basically we dont dispatch anything. Lets see broadcast . LD2 will broadcast, and write to F2, say its -2.5, And AD2, AD1 and ML1 will capture. Rat entry for LD2 is going to be invalidated. This concludes writing of the result. In this proc. we consider broadcast and dispatch, can't happen in same cycle. Thats why we did dispatch before broadcast. Similarly we can do dispatch first and then issue followed by broadcast. In which case we will not confuse to consider freshly issued instns for dispatch. So in cycle 6, we have issued, broadcasted, and nothing dispatched. Now lets consider what happens in cycle 7. From cycle 7 onwards we don't have any instn.s to issue , so we will no longer issue.

In cycle 7, both AD1 and ML1 can be dispatched, as they go to diff exe units. And there is nothing current executing, so we don't broadcast.

The sub we dispatched will broadcast in 9 and mul will do in 17.

In cycle 8 nothing happens except that the previous dispatched instns are executing.

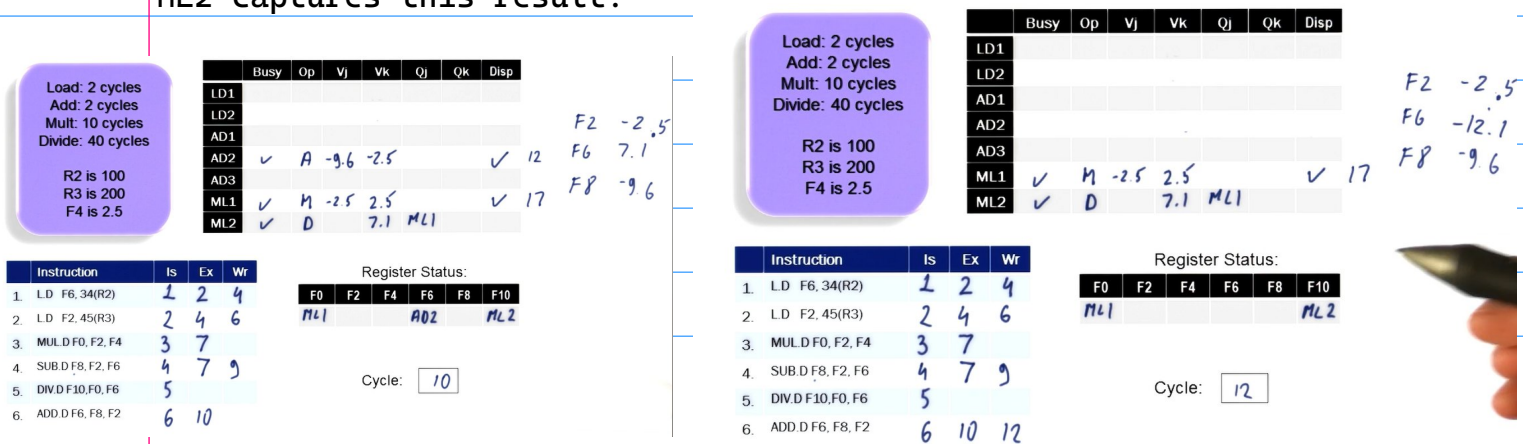
In cycle 9, Sub is gonna broadcast, its gonna find corresponding reg (that matches its tag) from rat and update in RF (as -9.6), also it will invalidate that rat entry. Also AD2 which is waiting for AD1 will capture. And we will free the RS .

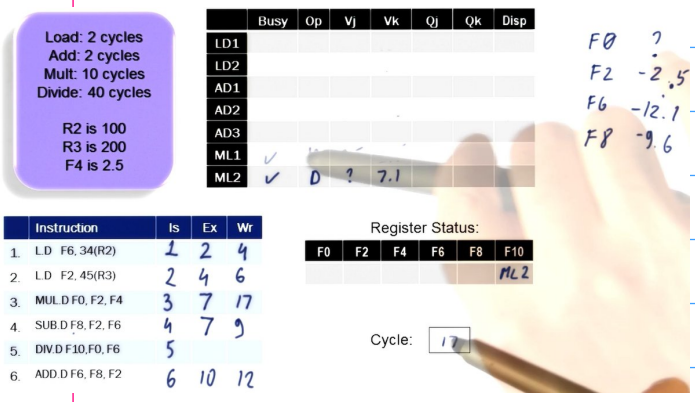


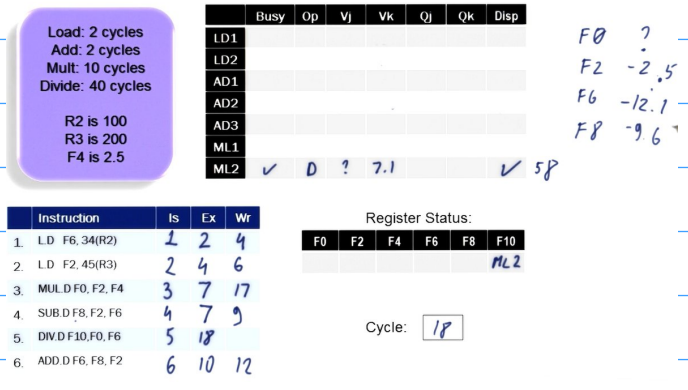
In cycle 10, AD2 will dispatch, and in cycle 12 it will broadcast/ write result. In cycle 11 only execution happens nothing else.

In cycle 12 , AD2 will broadcast result, we will update F6 to -12.1 and invalidate the F6 entry in RS. And invalidate AD2 RS.

Now nothing will really happen until cycle 17, when the ML1 is gonna broadcast. SO F0 get -6.25, and RAT and RS having this instn will invalid ML2 captures this result.



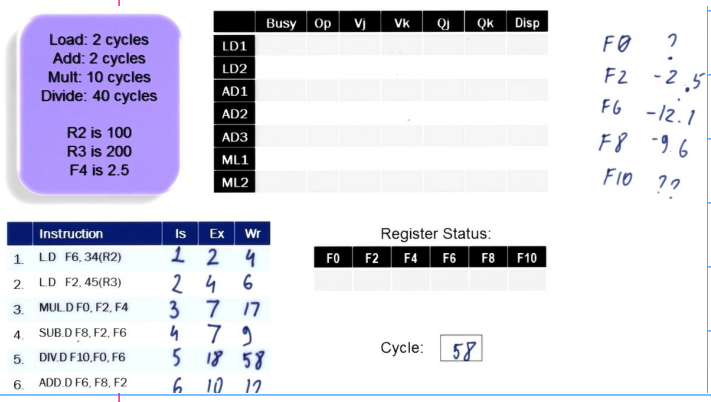


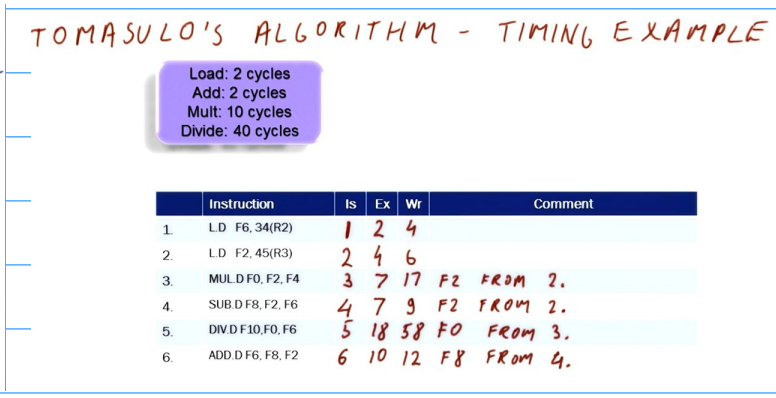


Finally in cycle 18, ML2 will be dispatched, and in 58, it will write back.

And in cycle 58, ML2 result will be broadcasted, written into F10, RAT holding ML2(rs) and RS holding this div instn will be invalidated, and nobody is waiting to capture.

And that concludes the example.





If we only need timing, we can do like shown above figure, given that the number of RS and exe units is given. We can mark Wr cycles tentatively and at the last we need to check if any two/more instr is broadcasting in sam cycle, if so , we need to postpone other broadcast and redo the later instn's timings.

TOMASULO TIMING QUIZ PART 1

LATENCY:
LD : 1 CYCLE
ADD : 1 CYCLE
MUL : 5 CYCLES

OF RES. STATIONS:
LD : 1
ADD : 2
MUL : 2

SAME-CYCLE:
ISSUE → DISPATCH : NO
CAPTURE → DISPATCH : NO
RS FREED → RS ALLOC : NO

	INST	ISSUE	DISP	WR
1	LD F6, 0(R2)	1		
2	MUL F2, F0, F1	2		
3	ADD F6, F2, F6	3		
4	ADD F6, F2, F6	4		
5	ADD F1, F1, F1			
6	ADD F1, F3, F4			

inst1 dispatches in C2 and writes in C3

inst2 dispatches in C3 and writes in C8

inst3 dispatches in C4 and writes in C5

inst4 disptaches in C5 and writes in C6

IF there were no dependencies

*inst1 dis in C2 and wr in C3

*inst2 dis in C3 and wr in C8

*inst3 dis in C9 and wr in C10

*inst4 dis in C11 and wr in C12

Part2 of the quiz is when do instns 5 and 6 do their issue, dispatch & wr

	INST	ISSUE	DISP	WR
1	LD F6, 0(R2)	1	2	3
2	MUL F2, F0, F1	2	3	8
3	ADD F6, F2, F6	3	9	10
4	ADD F6, F2, F6	4	11	12
5	ADD F1, F1, F1			
6	ADD F1, F3, F4			

IF there were no data & structural dependencies,

inst5 issue@C5 disp@C6 wr@C7

inst6 issue@C6 disp@C7 wr@C8

But theres no RS free for add instn.

inst5 issue@C11 disp@C12 wr@C13

inst6 issue@C13 disp@C14 wr@C15, we can't issue in 12 as a free entry in RS is not available for add.

In this lesson we have learned how proc. can rename and reorder instns to work around data dependencies. In the nxt lesson we will see what happens when we need to do something unexpected, like handle a div by 0, or a memory protection exception.